Understanding and creating data visualizations with LLMs

Sonja Gievska
Martina Tosevska
Dejan Ristovski

Faculty of computer science
and engineering - FINKI, UKIM

Abstract— **Chart generation and understanding is a problem that may seem quite easy to implement using LLMs, although large models with high reasoning capabilities can easily address this problem, smaller models should not be overlooked and deserve evaluation for their chart generation capabilities. Can these models understand user's needs of chart visualization from just plain visualization description, and can they also then generate a suitable chart matching their needs? This paper strives to answer these questions by testing small LLMs that are open source, such as the LLaMA family of models, the Deepseek r1 7B model and the newer Qwen 3 8B model. With specific prompt design and using state-of-the-art prompt techniques, the models are tested in different chart understanding and generation scenarios. The results reveal varying performance across the models, highlighting weaknesses in chart generation for some, while also demonstrating effective prompt techniques that improve outcomes in others.**

I. Introduction

With the growth of generative artificial intelligence and LLMs, an increasing number of everyday tasks and problems can be automated. One of the most frequent usages of LLMs is their ability to generate code and give reasoning for their actions. This also shows their incredible ability to understand any request and use their general knowledge to help in almost any scenario. While most models used for research have 70B+ parameters, the small models have been quite neglected. As LLMs are starting to be integrated into almost any system, the research for the capabilities of small models is starting to raise.

Chart creation is a common requirement for data exploration and reporting, so automating this process could significantly reduce the technical barrier for non-expert users. In this paper we focus on the chart understanding and creation for LLMs with small parameter size. We test the models in scenarios where they need to generate a chart based only on the user's input, even without specifying the chart type that needs to be generated. This way we plan to simulate a real scenario where a non-expert may interact with an AI agent for dataset visualization.

In contrast previous works on this subject (Pere-Pau Vázquez 2024, Guozheng Li et. al 2024, Zhongzheng Xu et. al 2024) were all testing models with high parameter size, such as GPT3 or GPT4. Vázquez (2024) examined whether state-of-the-art LLMs are ready to perform visualization tasks in practical contexts, while Li et al. (2024) provided a broad evaluation of visualization generation with large language models. Similarly, Xu et al. (2024) explored the ability of large models to perform low-level analytic tasks on SVG-based visualizations. While these studies highlight the potential of large LLMs to handle complex visualization and reasoning tasks, they do not address the performance of smaller, resource-efficient models that are increasingly relevant for deployment in everyday systems.

Our contributions are as follows:

1. We introduce a generated benchmark dataset of user visualization descriptions paired with expected chart types, generated using GPT-4 as a teacher model.
2. We test the models on two different python libraries used for chart generation tasks, matplotlib and plotly

3. We evaluate four small-parameter open-source models - LLaMA 3.2 3B, LLaMA 3.1 8B, Deepseek R1 7B and Qwen 3 8B - on chart generation tasks with different prompting strategies.

4. We provide reasoning prompts and prompt engineering for improving chart generation quality for lightweight LLMs.

II. Related Works

The problem of creating data visualizations with LLMs has been attracting researchers for a long time. For example Pere-Pau Vázquez 2024, tested both ChatGPT3 and ChatGPT4 in creating a number of different charts. Their tests for the LLaMA2, Code LLaMA2 and Mixtral models in their 7B parameter versions showed a lot of hallucinations and errors, which shows that the older models were not suitable for these type of problems. They also tested 3 different python libraries and 24 different number of chart types. Their results showed that the larger models have great results for most types of charts, which 16 out of 24 were generated successfully for ChatGPT3 and 19 out of 24 for ChatGPT4.

Other researchers such as Guozheng Li et. al 2024 decided to use different techniques for generating charts. This paper showcased the ability of GPT-3.5 to generate Vega-Lite specifications that can be used to display visualizations. A lot of prompt techniques were used to test the model, such as Chain-of-Thought, Rationale Engineering and Problem Decomposition. But the most notable in the results was the few-shot prompting technique that showcased better results over the zero-shot prompting. Their evaluation metrics consisted of a combination of the ground truth visualization, and Vega-Lite specification. The results were not satisfactory, probably because the models did not understand the Vega-Lite specification and format.

The researchers in Zhongzheng Xu et. al 2024 tested GPT-4-turbo-preview for the generation of SVG code for data visualizations. The model was tested with zero-shot prompting for 10 different tasks such as, retrieving values, filtering, sorting, clustering, and computing derived metrics. The model displayed great results for clustering or retrieving labeled values, but struggled for tasks that required mathematical reasoning or computation.

The LLaMA 3 family of models was first introduced in the paper Aaron Grattafiori et. al 2024. The models supported coding, reasoning, and tool usage, making them perfect for testing them in chart generation techniques. The LLaMA 3.2 3B parameter model is mostly meant to be used in mobile devices or edge devices, but still has all the benefits of the higher parameter models in the same family.

The Deepseek R1 7B model was introduced in Daya Guo et. al 2025 and was trained using pure reinforcement learning methods. The model became popular for its incredible reasoning capabilities that are comparable with state-of-the-art models such as OpenAI's o1 series models. This shows a great potential in the chart generation and understanding domain.

The Qwen3 8B model was introduced in An Yang et al., 2025 as part of the Qwen3 family. The model balances both efficiency and performance, making it suitable for research and deployment. It features extended multilingual support across 119 languages and long context handling up to 128K tokens. The model also displays strong performance in coding, math, reasoning, and tool use, making it a versatile general-purpose LLM.

III. Overview of techniques

Since the goal is to test the model's ability to understand the user needs and generate a chart, a proper dataset and prompt techniques were needed. All techniques for chart generation, including the prompts, models, datasets and evaluation metrics are described in the following sections:

A. Selection of charts and Dataset

We used both synthetically generated dataset and publicly available dataset for testing the model's capabilities:

1. The synthetically generated dataset consists of user visualization descriptions and the expected chart type. The dataset consists of 30 descriptions, each generated with GPT 4 that is used as a teacher LLM. Since the models used are of smaller parameter size, a bigger model that has been trained on significantly higher data can be used as a baseline to create a testing dataset.

2. The second dataset is provided by pyspark-ai (Xinrong Meng et. al) that consists of user visualization descriptions, along with ground truth code samples and plot descriptions. These plot descriptions consist of data that is displayed for each axis in the chart, along with the chart type that is generated. This dataset was used for the ability to evaluate the generated charts, without comparing the charts directly as demonstrated with the exact match technique in other papers (Zhongzheng Xu et. al 2024).

For the synthetic dataset tests, the dataset used for the data that the model generates charts from is a public dataset about sleep health and lifestyle (Laksika Tharmalingam). On the other hand, the datasets for the public dataset are provided in the github repository (Xinrong Meng et. al), along with the ground truth code. The chart types used in the synthetically generated dataset are: Histogram, Bar chart, Scatterplot, Pie chart and Line chart, while for the public dataset Box plots are also used.

B. Selection of models

Four models in total were selected to be used in this research. The models were selected based on various characteristics such as parameter size, intended use, reasoning capabilities etc. The LLaMA models are run using Ollama on an RTX 4060 GPU, utilizing the Q4_K_S quantized versions, while the Deepseek and Qwen models were used in their base form without any quantization.

- Llama 3.1 8b: A relatively lightweight model fine-tuned for tool calling, representing the latest in the LLaMA 8B family.
- Llama 3.2 3b: This model is intended for usage in mobile phones based on the small parameters and lightweight architecture, while still utilizing all characteristics of the higher parameter models in its family
- Deepseek 7b: One of the more popular open-source models, it demonstrates strong reasoning capabilities compared to other models such as LLaMA or OpenAI's o1 models
- Qwen3 8b: Used as the newest model, also posing great chart generation capabilities for the Pyspark python library.

C. Chart Generation Techniques

Several chart generation techniques were used to test the models understanding of chart creation. For each model, a general system message was used to direct the model to respond in a structured manner. Using custom tags like <chart-code></chart-code> to extract the code snippets and chart types that the model generates. Some prompt engineering was also used to direct the model not to generate a sample dataset, but to use the already loaded one.

In the user message, a sample of the dataset was given to the model as reference of the types and names of columns in the dataset. Besides the sample example, each user message was also altered based on the tasks that the models needed to do. This paper covers 3 chart generating tasks including:

- Asking the model to pick and generate a chart

The model is given a vague description of what needs to be visualized, without giving any indication of what chart type is needed. This tests the model's ability to decide which chart is the best to visualize the problem. For the public dataset, the chart descriptions annotated with WithoutPlotType complexity were used.

- Asking the model to pick and generate a chart + Reasoning

The model is prompted to generate its thinking and then generate a chart. Prompt engineering was needed for the model to give structured output, so one shot prompting was used. The example given was a sample output of how the model should generate its results. This improved the models results by a significant amount.

- Giving the model chart type and asking to generate it

The model is given a description and the type of chart that it needs to generate. This gives the model less freedom to generate different charts, so that the model's chart code generating techniques are tested. For the public dataset, the chart descriptions annotated with WithPlotType complexity were used.

By structuring the evaluation in this way, we highlight not only the models' chart generation skills, but also the impact of prompt design and reasoning techniques on improving performance. A sample of one used prompt can be seen in Figure 1.

"system":
You are a text-to-chart generating model. Always generate the charts using python and matplotlib.
Always use the dataset example given that is already loaded with pandas as variable *df*. Do not generate code for loading the dataset!
Do not hallucinate examples or generate sample data. Generate only one code. Respond only with the code.
Generate the chart type in the beginning between tags <*chart-type*></*chart-type*>.
Generate the code between tags <*chart-code*></*chart-code*>

"user":
Show the relationship between sleep duration and quality of sleep

Example data:

Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep
1	Male	27	Software Engineer	6.1	6
2	Male	28	Doctor	6.2	6
3	Male	28	Doctor	6.2	6
4	Male	28	Sales Representative	5.9	4
5	Male	28	Sales Representative	5.9	4

Figure 1. Example user and system prompt for task 'Asking the model to pick and generate a chart'

D. Evaluation metrics

A simple evaluation metric was used for evaluating the model's responses using the synthetic dataset. Since models with small parameter size were used, a lot of the responses were not in the correct format and the extraction of code snippets was failing. Thus, the models were evaluated using the following evaluation approaches:

1. Hit vs. Miss Accuracy
    - A simple metric that determines whether the model generated a complete code snippet that compiles, and whether the generated type is matched with the expected chart type
    - The overall hit percentage is measured based on the number of hits, divided by total number of chart descriptions
2. Manual Quality Scoring
    - In addition to type-level accuracy, a manual scoring techniques was used that determines if the model captured the chart description with another chart type
    - Each response was assigned a score with three-point scale:
        - **0** - The model failed to generate a correct chart, or no chart was displayed.

- **1** - A chart was generated, but it was not complete or could have been better represented with an alternative chart type.
- **2** - The generated chart matched the user's problem.

Only manual quality score was used for the chart generation with given chart type problem, since the hit score is based on the models chart type selection capability.

Besides the hit vs. miss accuracy metric, the generated results were also evaluated based on the ground truth data for the public dataset tests. Each ground truth sample was compared with the generated chart data, and an average score was used based on if the chart data part was a hit or a miss. Each failed code generation sample was scored with 0.

IV. Results

In the following section, the results are displayed along with a few remarks on how the model handled the structured output generation for the synthetic dataset.

	LLaMA 3.2 3B		**LLaMA 3.1 8B**		**Deepseek 7B**	
	Hit vs. Miss	Manual Scoring	Hit vs. Miss	Manual Scoring	Hit vs. Miss	Manual Scoring
Ask generation	**0.3**	**0.5**	**0.63**	**0.8**	**0.6**	**0.48**
Ask generation + Reasoning	**0.76**	**0.82**	**0.9**	**0.92**	**0.57**	**0.67**
Give chart type	/	**0.95**	/	**0.96**	/	**0.58**

Figure 2. Results from all tasks and used models for synthetic dataset

	LLaMA 3.2		**LLaMA 3.1**		**Qwen3**	
	Hit	Score	Hit	Score	Hit	Score
WithoutChartType	**0.14**	**0.15**	**0.48**	**0.39**	**0.54**	**0.46**
WithoutChartType + Reasoning	**0.25**	**0.21**	**0.45**	**0.39**	**0.5**	**0.41**
WithChartType	**0.31**	**0.23**	**0.8**	**0.66**	**0.73**	**0.62**
Reasoning + Pandas	**0.41**	**0.35**	**0.59**	**0.47**	**0.63**	**0.53**

Figure 3. Results from all tasks and used models for public dataset

- Asking the model to pick and generate a chart
  - LLaMA 3.2 3B - The model usually chooses to generate the simplest chart based on it's knowledge, not trying to generate something that matches the users problem better. Because of this the Hit vs. Miss score is quite low. On the other side, based on the manual scoring, the model still gives a chart with a different type that still represents the users problem.
  - LLaMA 3.1 8B - This model gives quite an improvement over the smaller 3B model. The model generates the expected chart type and also has a high manual score. The model also makes less mistakes, with only one fail in the code generation part.
  - Deepseek 7b - While the model almost matches the Hit vs. Miss score of the LLaMA 3.1 model, the manual score is the lowest of the models. This is because it mostly generates a response that does not display the chart, or generates code that does not compile.
- Asking the model to pick and generate a chart + Reasoning
  - LLaMA 3.2 3B – Big improvements were seen for this model with both scoring techniques. The model gave much more structured outputs that could be parsed and executed correctly. Also, the model generated more versatile charts that aligned with the expected one from the testing dataset.
  - LLaMA 3.1 8B – This model also saw a lot of improvement with the thought generation technique. The Hit vs. Miss metric was a lot better, most likely because the model now gives more structured outputs.
  - Deepseek 7b – On the opposite, this model performed with worse precision. This might show some bigger bias in the models training, that prevents it from giving structured outputs. The Manual score has increased, although not by a large margin.
- Giving the model the cart type
  - LLaMA 3.2 3B – With this technique, the model has displayed its chart generation techniques and now gives very high scores. This means that the model is much better in generating chart code, than determining the chart type from a description.
  - LLaMA 3.1 8B – This model follows the results from the previous with a small improvement, showing that parameter size does not matter for this type of problem.

- Deepseek 7b – Following the model's results from the previous tasks, not much improvement was seen with the reasoning technique. This could point to a problem of generating chart code that the model may not be suited for.

There was a problem when testing the DeepSeek R1 model on the public dataset. The model displayed poor results for generating Pyspark code, and most of the code did not execute. As a result, a change was made where a new model, Qwen3 8B, was used instead. This is a newer model than DeepSeek and it showed much better results.

For the smaller LLaMA 3.2 3B model, the results were disappointing. The model also struggled to generate Pyspark code and produced mostly unusable outputs. This may indicate that models with smaller parameter sizes do not have enough capacity to effectively handle code generation across multiple Python plotting libraries. In contrast, the larger 8B model showed significantly better results than the smaller one, even though much of the Pyspark code still contained errors. Overall, Qwen3 delivered the strongest performance on the public dataset, with the highest success rate for code generation.

Most of the scores for the models are very close to the hit rate, indicating that when the model generates proper Pyspark code, it will almost always produce a correct plot. This shows that the models mainly need to improve their generation capabilities for different Python charting libraries. A lot of different prompts were also used, but if the model improved in handling one error, a different occurred and the results came as even worse. Thus, a simple prompt was decided to be used since it gave the best results.

For the last tests the models were tested on the pandas library instead of Pyspark, which improved each models performances. This was not aligned with the ground truth code provided from the datasets, where Pyspark was used, but it still generates results that can be compared with the ground truth chart data. This indicates that the smaller models are mostly trained on the more popular python libraries for data manipulation and visualization such as the pandas and matplotlib libraries.

V. Conclusion

This research experimented chart generation techniques with various LLM models with small number of parameters. While a lot of work was spent prompting the model to generate a structured

output, the results show that careful prompt design and reasoning instructions significantly influence the performance.

The LLaMA models show high results when prompted with reasoning techniques, showcasing their capability to understand users needs and generate suitable charts based only on user visualization descriptions. The models also showed very high scores when only generating chart types and demonstrated that parameter size does not matter for this kind of task. However, when using another charting library, a gap in the models knowledge has occurred. Further testing may include the 70b models to see if this is a problem for the entire LLaMA family of models, or just the models with small parameter size.

The Deepseek model showed a lot of problems with these tasks. Even when prompted to give reasoning, the model struggled to generate a structured output and suitable charts from the given user descriptions. Providing the expected chart type did not improve the models results either, possibly highlighting its weakness in generating chart code. The model also displayed poor performance when using the Pyspark library and the outputs were unusable.

On the other hand, Qwen3 displayed great performance for the Pyspark dataset showcasing its ability to handle different python libraries. This may indicate that the model was mostly trained on code samples and code generating instructions. Further research may try to test the models with higher parameter size to see if even more satisfying results are shown.

Therefore, even though chart generation is a task well suited for LLMs, proper model and prompt engineering is needed to get satisfying results. Using the correct python libraries and techniques are also important for getting good results, even more so when using models with small parameter sizes.

VI. References

[1] Pere-Pau Vázquez 2024, "Are LLMs ready for Visualization?"

[2] Guozheng Li et. al 2024 "Visualization Generation with Large Language Models: An Evaluation"

[3] Zhongzheng Xu et. al 2024 "Exploring the Capability of LLMs in Performing Low-Level Visual Analytic Tasks on SVG Data Visualizations"

[4] Aaron Grattafiori et. al 2024 "The Llama 3 Herd of Models"

[5] Daya Guo et. al 2025 "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning"

[6] An Yang et. al 2025 "Qwen3 Technical Report"

[7] Xinrong Meng et. al "Text-to-Plot Benchmark"

[8] Laksika Tharmalingam "Sleep Health and Lifestyle Dataset"